DAP 391 - Group 4

HypeStock Project Final Report

Abstract

HypeStock is an AI-powered stock analytics analysis and prediction platform developed with Python, Redis (Sanfilippo, 2009) [1] and TimescaleDB (Timescale, 2017) [2] - Powered by vnstock (Thinh, 2022) [3]

Author's Note: This report provides a comprehensive overview of the project's objectives, system architecture, and evaluation results. For a deeper, exhaustive breakdown of the mathematical formulations, data engineering pipelines, and hardware-aware training loops, please refer to the following document: *"Technical Overview of a Production-Grade AI-Powered Stock Analytics Platform"*.

Table of Contents

1. **Introduction**
   - 1.1 Background
   - 1.2 Problem Statement
   - 1.3 Project Aim
   - 1.4 Project Scope
     - 1.4.1 Core Platform & Market Analysis (Normal Function)
     - 1.4.2 Predictive Modeling & Inference (Prediction)
     - 1.4.3 Conversational Intelligence (AI Chatbox)
   - 1.5 Constraints
   - 1.6 Objectives

2. **Business and Data Understanding**
   - 2.1 Target Users and Use Case
   - 2.2 Dataset Overview

3. **Data Collection and Preparation**
   - 3.1 Data Collection
   - 3.2 Data Cleaning
   - 3.3 Feature Engineering

4. **System Architecture**
   - 4.1 Overview
   - 4.2 Data Pipeline
   - 4.3 Component Integration

5. **Methodology: Predictive Modeling**
   - 5.1 Model Architecture
   - 5.2 Training Process

6. **Methodology: AI Chatbot Agent**
   6.1 Agent Logic
   6.2 Knowledge Retrieval
   6.3 Prompt Engineering
   6.4 Safety & Guardrails

7. **Implementation & Tech Stack**

8. **Evaluation & Results**
   8.1 Predictive Accuracy
   8.2 Agent Performance
   8.3 System Usability

9. **Discussion**
   9.1 Analysis of Findings
   9.2 Challenges
   9.3 Limitations

10. **Conclusion & Future Work**
    10.1 Summary
    10.2 Future Work

11. **References**

1. Introduction

1.1 Background

The evolution of algorithmic trading has democratized access to high-frequency market data, providing retail investors with analytical resources once exclusively reserved for institutional desks. However, financial markets remain inherently complex environments characterized by non-stationary dynamics, volatility clustering, regime shifts, and frequent structural breaks. While the accessibility of stock data has increased exponentially, the analytical burden of processing this multivariate history has shifted directly onto the individual user.

1.2 Problem Statement

Despite the abundance of accessible market information, a significant gap persists between raw data availability and actionable user insights. The principal challenge is converting heterogeneous raw market observations into stable predictive signals that can survive real-world data drift and delayed ingestion.

Furthermore, while modern users expect conversational analysis, general-purpose AI tools frequently struggle in financial contexts, often fabricating explanations if they are not strictly grounded in deterministic data retrieval.

1.3 Project Aim

The HypeStock project aims to develop an end-to-end production ecosystem that unifies a high-throughput machine learning pipeline with a conversational AI agent. The primary goal is to deliver a platform that produces calibrated 7-day forecasts while ensuring all conversational outputs are strictly grounded in deterministic database telemetry.

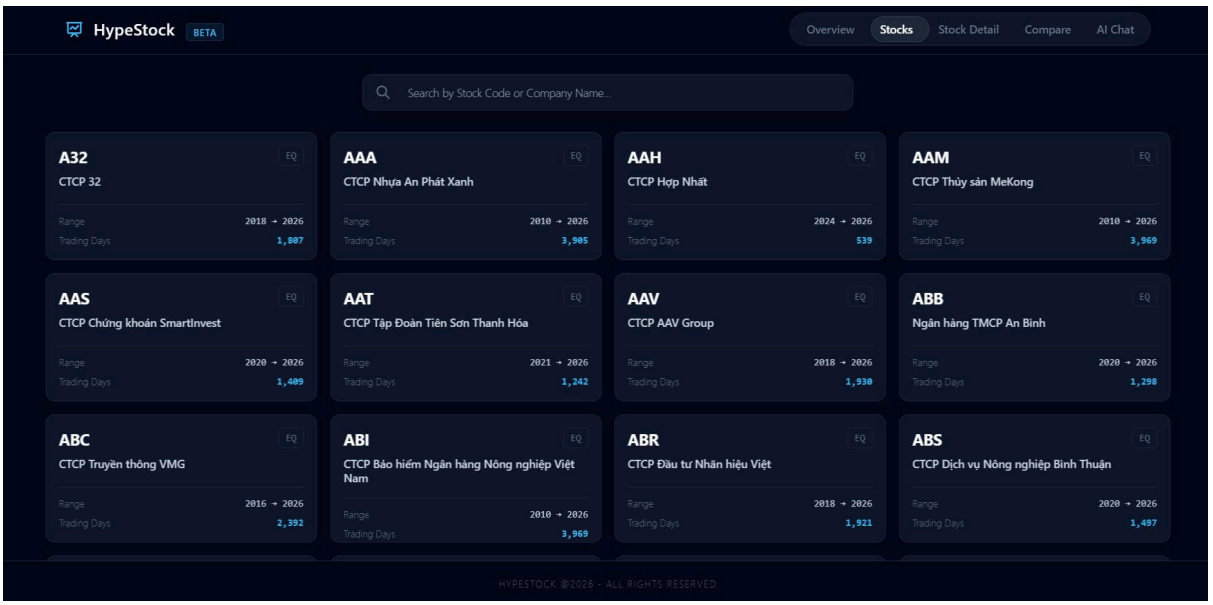
1.4 Project scope

The implementation of HypeStock is divided into three primary functional domains, ensuring a balance between traditional financial tools, advanced forecasting, and conversational intelligence.

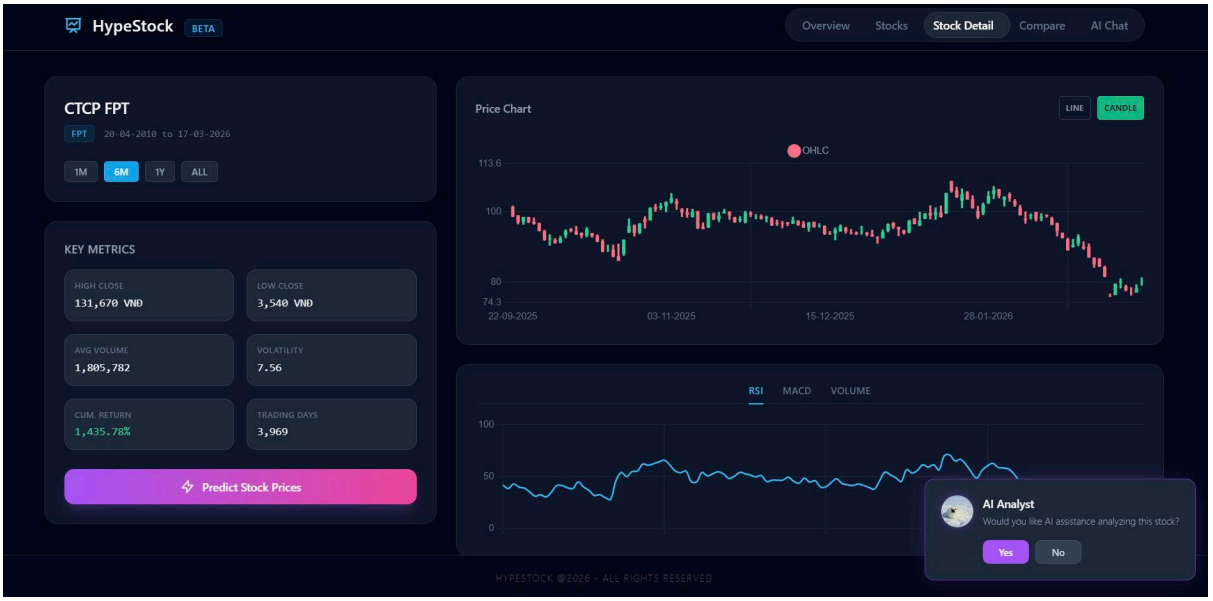
1.4.1 Core Platform & Market Analysis (Normal Function)

This domain covers the standard analytical tools required for baseline market observation and data management.

- Data Integrity & Management: Implementation of a high-resolution List of Data (Table Format), providing a transparent view of the raw market observations and technical indicators stored in TimescaleDB.



- Detail Analysis: Comprehensive single-stock views that aggregate historical performance metrics for retail-level interpretation.

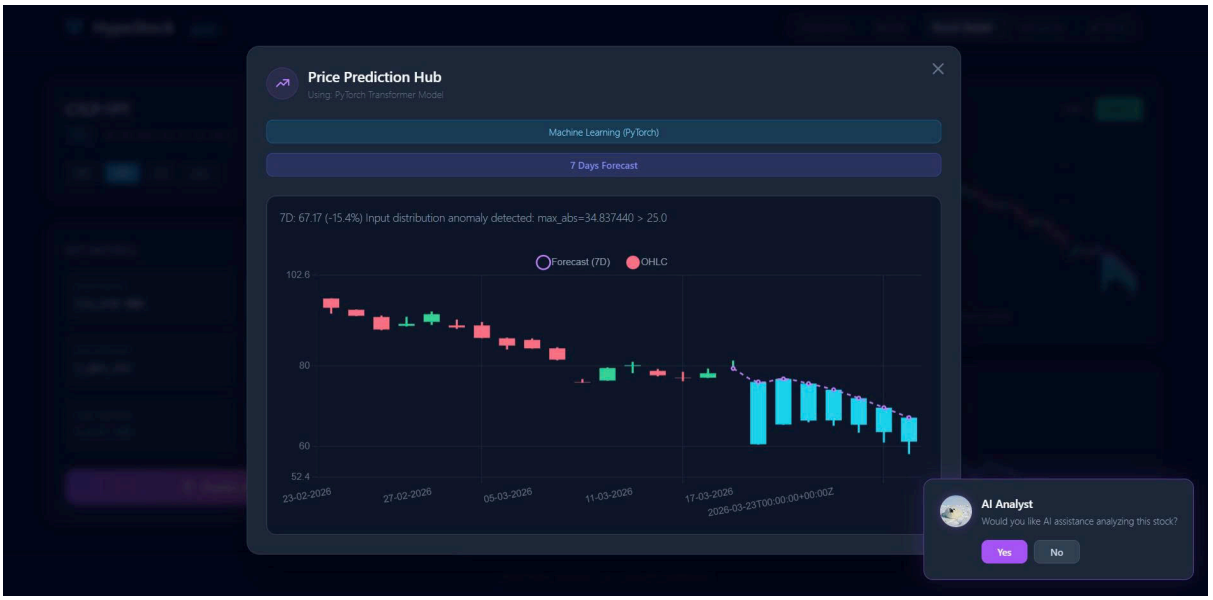


- Stock Comparison Suite: A module allowing users to perform cross-sectional analysis using the Stock Compare Select Function and the Stock Compare Graph to visualize correlations between instruments.

1.4.2 Predictive Modeling & Inference (Prediction)

The predictive scope is centered on the Prediction Hub, which serves as the visual interface for the system's machine learning backbone.

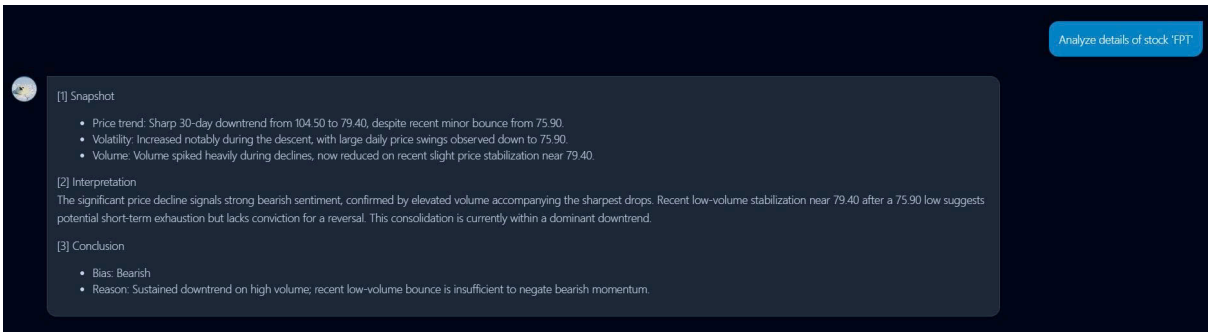
- Hybrid Forecasting: The scope includes the deployment of a Transformer-GRU sequence model that generates 7-day forward log-return targets.



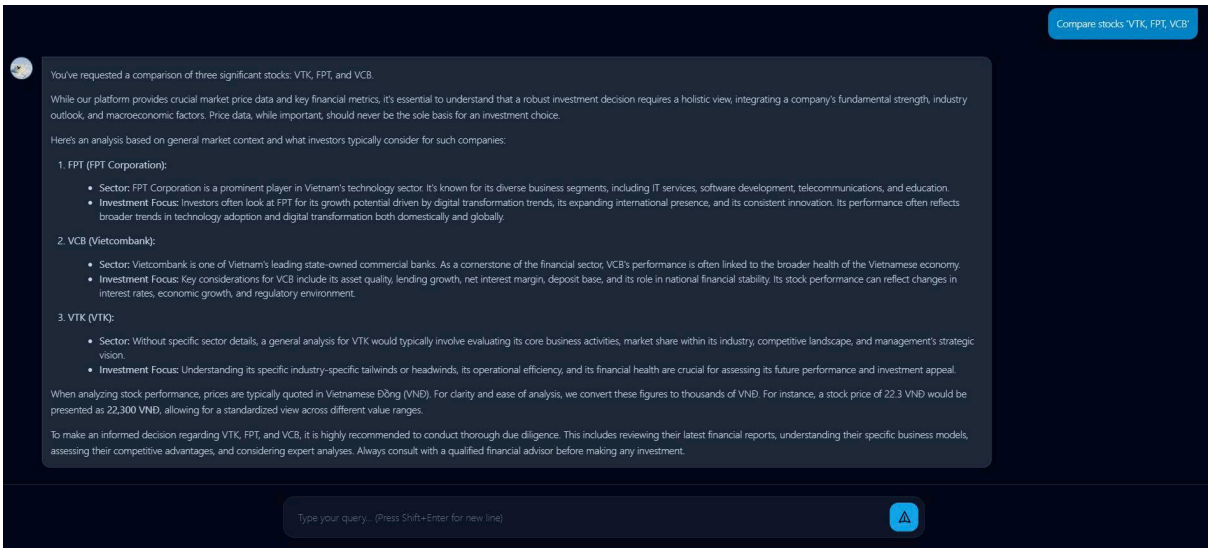
- Confidence Telemetry: Visualization of prediction stability (mean and standard deviation) to provide users with quantitatively calibrated forecasts rather than simple price guesses.
- Asynchronous Processing: Utilizing Celery to handle the heavy computational load of generating these predictions without impacting the responsiveness of the web client.

1.4.3 Conversational Intelligence (AI Chatbox)

The AI scope focuses on bridging the gap between dry financial data and natural language interaction.



- Grounded AI Response: Implementation of a chat interface where AI Chatbox Responses are strictly derived from deterministic database queries via PandasAI, preventing hallucinations.
- Session Memory Management: A specialized subsystem that semantically compresses interaction history (see AI Response components), allowing the agent to maintain context throughout long analytical sessions.
- Intelligent Routing: A logic layer that determines whether a user's query requires a statistical summary, a predictive outlook, or a direct database retrieval.



1.5 Constraint

The development and deployment of the HypeStock platform are governed by the following technical and operational constraints:

- **Hardware Limitations:** The system must be optimized for local deployment on laptop-class hardware (specifically a Ryzen 7 7435HS and RTX 4060 with 16GB RAM), requiring mixed-precision training and gradient accumulation to manage VRAM pressure.
- **Target Horizon:** Although the model architecture is theoretically capable of multi-horizon outputs, deployment is strictly constrained to a **7-day forecast** to minimize gradient interference and ensure optimization stability.
- **Data Integrity:** To prevent "data leakage," all features must be generated using chronological sliding windows, ensuring the model never "sees" future information during the training phase.

- **Operational Safety:** The AI agent is restricted from providing speculative financial advice; all quantitative responses must be verified against deterministic database queries prior to release.

1.6 Objectives

- **Responsive web interface:** Create a low-latency, interactive user experience that can deliver real-time signal updates and asynchronous heavy tasks (like retraining and backfilling) without bottlenecking the client.
- **Time-Series Prediction Model:** Architect and train a hybrid PyTorch sequence model specialized for predicting a 7-day forward log-return target to stabilize scale across various instruments.
- **Integrate a data-aware LLM agent:** Implement an intelligent routing layer using the Gemini API and PandasAI to enable natural language querying that remains safely grounded in deterministic database schemas to prevent hallucinations.

2. Business and Data Understanding

2.1 Target Users and Use Case

The proposed system is primarily designed to support **retail investors**, who often face challenges in interpreting complex financial data and making timely investment decisions. Unlike institutional investors, retail users typically lack access to advanced analytical tools and domain expertise.

Accordingly, the system aims to address the following user needs:

- **Efficient interpretation of market trends:**
  Enable users to quickly understand price movements and underlying patterns without requiring deep technical knowledge.

- **Short-term predictive insights for decision-making:**
  Provide data-driven forecasts to assist users in making informed short-term investment decisions.

- **Accessible explanations of complex data:**
  Translate analytical outputs and model predictions into intuitive, human-readable insights through interactive components.

2.2 Dataset Overview

The dataset utilized in this study is sourced from financial market data providers and consists of structured historical trading records. The primary data components include:

- **Historical stock prices and Trading volume:**
  Open, high, low, and close (OHLC) values representing daily price movements and the number of shares traded, reflecting market activity and liquidity.
- **Derived technical indicators:**
  Engineered features capturing trends, momentum, and volatility, which enhance the model's ability to learn meaningful patterns.

Metric	Functional Category	Analytical Role in Model Training
Open, High, Low, Close (OHLC)	Raw Price Action	Defines the fundamental price boundaries for each sliding window observation.
Volume	Liquidity Metric	Acts as a weight for price conviction, helping the Transformer encoder distinguish between noise and significant trends.
MA20, MA50, EMA20	Trend Indicators	Establishes the baseline directional bias and smooths high-frequency fluctuations.
RSI, MACD	Momentum Oscillators	Captures overbought/oversold states to prevent the model from over-leveraging during extreme regime transitions.
Volatility, ATR	Risk & Range Metrics	Calibrates the system's sensitivity to price swings, directly influencing the **Volatility Alignment** ($\mathcal{L}_{vol}$).
Daily_Return_1d	Real-time Performance	Provides the immediate rate-of-change data required for the GRU decoder's temporal state updates.
Lagged_Return (t1, t3, t5)	Autoregressive Features	Supplies historical "memory" to the model, reducing dependence on immediate price data and improving sequence stability.
Dist_from_MA50	Mean-Reversion Metric	Measures the stretch from the long-term trend, indicating potential price exhaustion or trend breakout points.

Key characteristics:

- **Time-series structure:**
  The data is sequentially ordered by time, enabling the modeling of temporal dependencies and trends.

- **Multivariate feature space:**
  Multiple financial variables are incorporated simultaneously, allowing for a richer representation of market dynamics.

- **Daily frequency across multiple stocks:**
  Observations are recorded on a daily basis for a range of stocks, providing both temporal depth and cross-sectional diversity.

3. Data Collection and Preparation

3.1 Data Collection

The system begins with the acquisition of historical market data using the *vnstock* (Thinh, 2022) [2] library, which provides programmatic access to Vietnamese stock market data. This stage focuses on collecting raw financial observations in a structured and reproducible manner to support downstream analysis and modeling.

The primary data collected consists of **OHLCV (Open, High, Low, Close, Volume)** records for selected equity symbols over a specified time horizon. Each record is indexed by trading date and ticker symbol, forming a time-series dataset that reflects daily market activity.

Data retrieval is performed through API-based queries, ensuring consistency and scalability in data ingestion. The collected data is stored in a relational time-series database, where each observation is uniquely identified by its symbol and timestamp. This structure facilitates efficient querying, chronological ordering, and integration with subsequent processing stages.

To ensure data quality, basic validation steps are applied during ingestion, including:

- verification of timestamp continuity

- handling of missing or non-trading days due to market closures

- removal of duplicate or inconsistent records

By maintaining the raw dataset in an immutable format, the system preserves data integrity and enables reproducibility of experiments. This collected dataset serves as the foundational input for further preprocessing, feature engineering, and model development stages.

3.2 Data Cleaning

Data preprocessing is a critical step to ensure the quality, consistency, and reliability of the dataset prior to analysis and model training. The following procedures were applied:

- **Handling missing values:**
  Missing observations were identified and treated using appropriate imputation or removal strategies to prevent bias and maintain data integrity.
- **Timestamp alignment:**
  Temporal records were standardized and aligned to ensure consistency across all variables, enabling accurate time-series analysis.

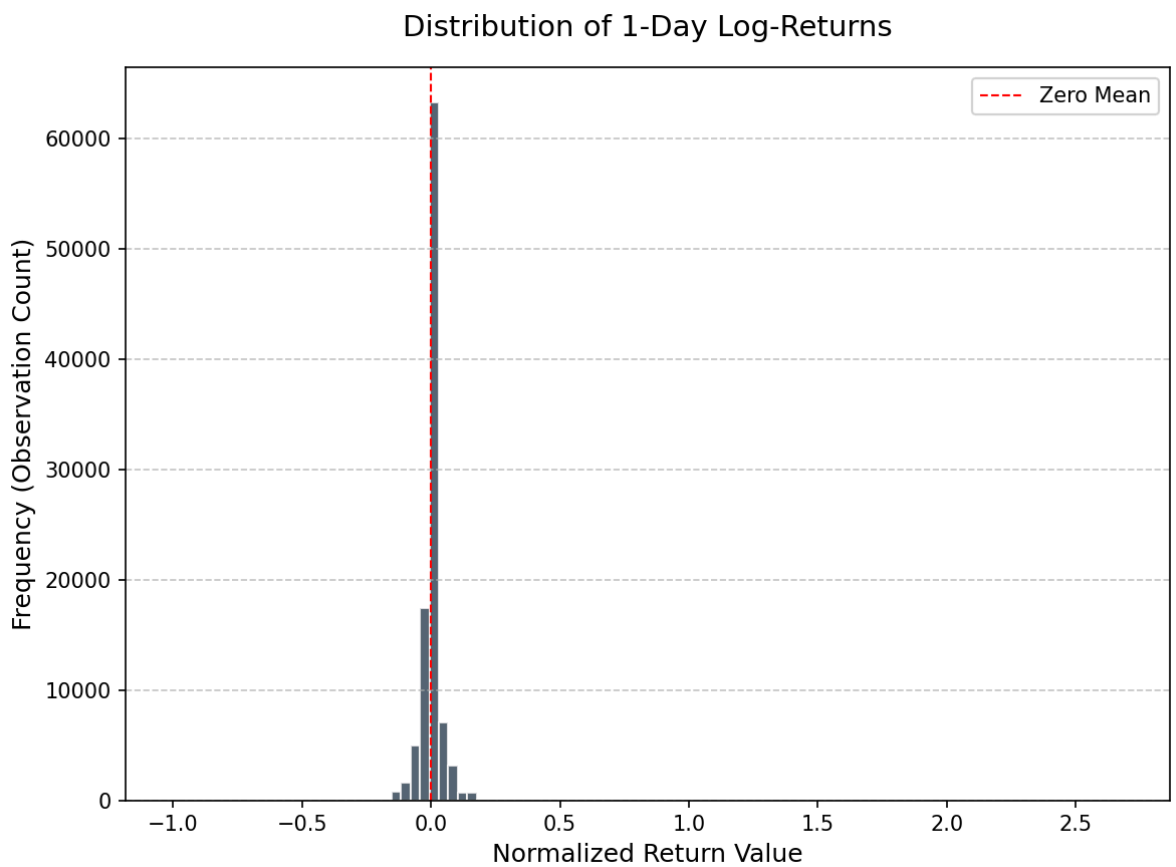
- **Removal of inconsistent records:**
  Erroneous, duplicate, or anomalous entries were detected and excluded to reduce noise and improve the overall quality of the dataset.

```
--- [BEFORE] Raw OHLCV Structure (df_default) ---
         open  high   low  close  volume
3903271  15.9  17.2  13.9   15.0     400
3903272  14.2  14.2  13.5   13.5    2300
3903273  14.2  15.5  12.0   12.0     400
3903274  13.6  13.7  12.3   12.3     300
3903275  13.9  14.0  13.9   14.0     200

--- [AFTER] Processed Feature Tensors (df_process) ---
           MA20    MA50    EMA20      RSI  ...  Lagged_Return_t1  Lagged_Return_t3  Lagged_Return_t5  Dist_from_MA50
3903271  15.635  13.448  14.6838  51.4702  ...            0.1603            0.1048            0.0821          0.1035
3903272  15.435  13.502  14.5711  46.9911  ...           -0.0132           -0.0438           -0.1448         -0.0001
3903273  15.140  13.530  14.3262  42.9646  ...           -0.1000            0.1603            0.1048         -0.1275
3903274  14.900  13.556  14.1332  43.9982  ...           -0.1111           -0.0132           -0.0438         -0.1021
3903275  14.780  13.600  14.1205  49.5745  ...            0.0250           -0.1000            0.1603          0.0286

[5 rows x 12 columns]
```

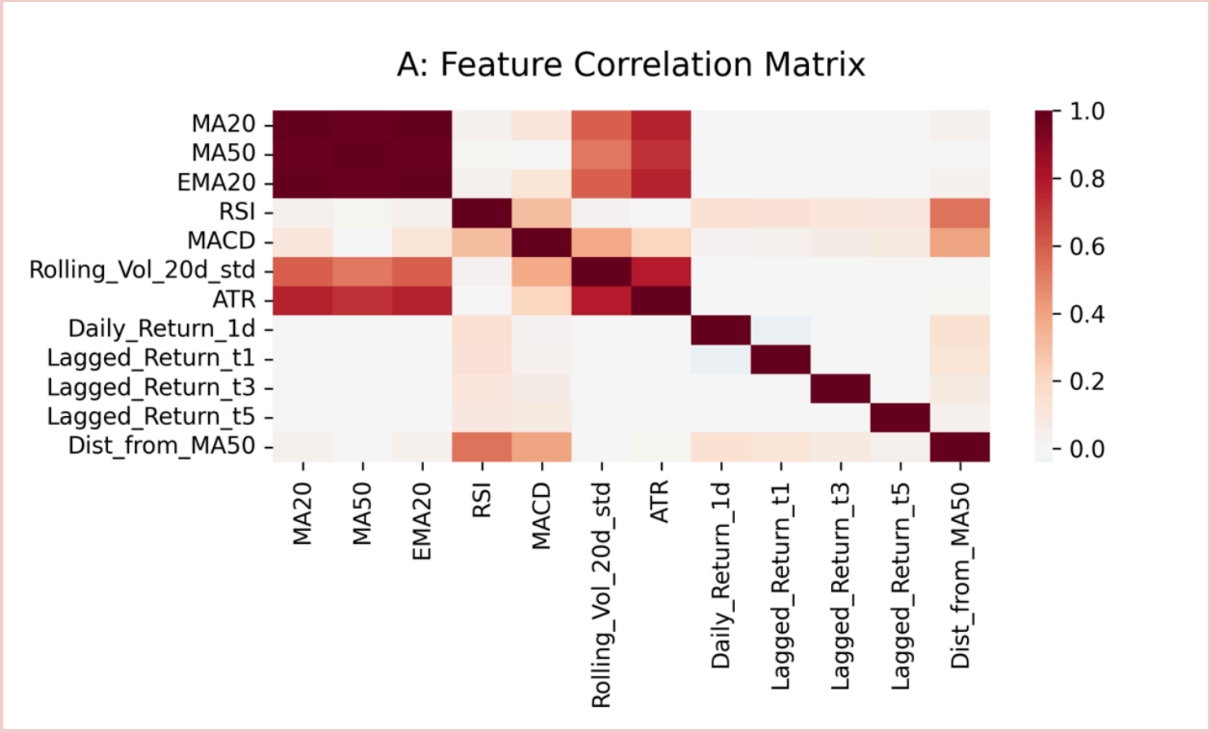
3.3 Feature Engineering

Feature engineering transforms the raw market candles into regime-sensitive predictors.

- **Technical Indicators:** Each timestamp is represented as a feature vector containing price transforms, volume dynamics, realized volatility estimates, liquidity-aware terms, and momentum terms, including standard indicators like RSI and MACD. Feature normalization is performed per feature using only training statistics to prevent data leakage, utilizing robust median-IQR scaling selectively for heavy-tailed variables.
- **Target Variables:** While time-series forecasting often involves predicting immediate short-term target variables like Close_{t+1}, this production architecture is specialized to output a single 7-day forward target. The base target is defined as a forward log-return, a mathematical design choice that stabilizes scale and improves optimization behavior across instruments with varying price levels.

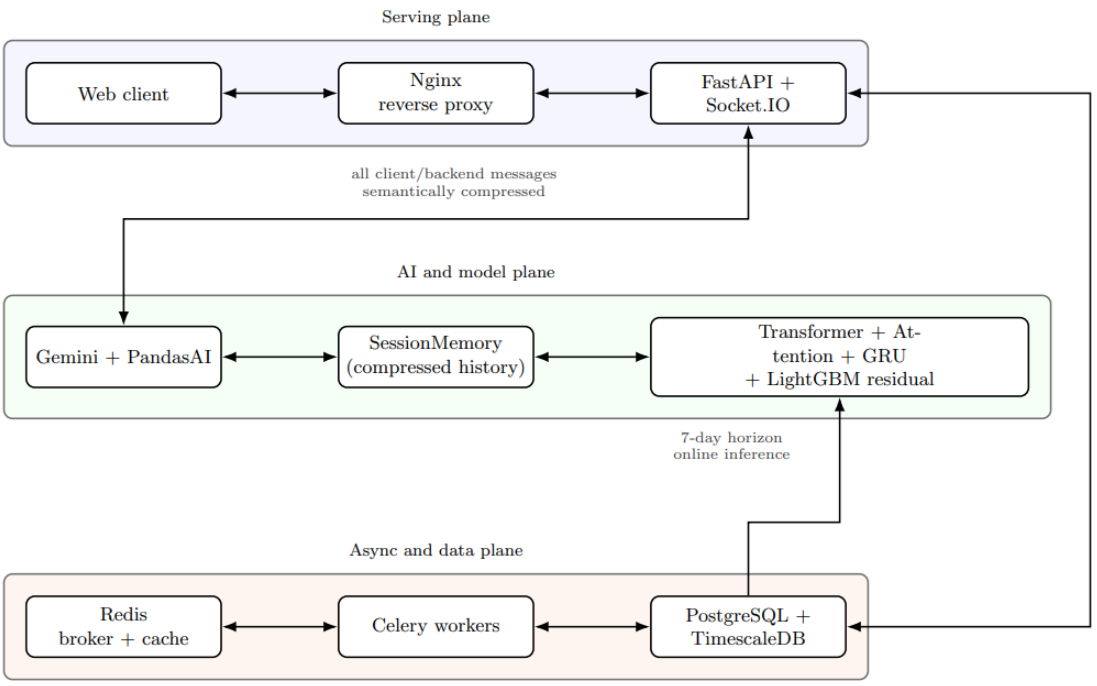


Shows the frequency of daily price changes. The zero-centered, stationary profile confirms successful normalization, allowing the model to process different stocks on a single mathematical scale.



Visualizes the relationships between all 17 features. Low correlation between momentum (RSI) and lagged returns (t-1) ensures a diverse, non-redundant input space for the Transformer model.

4. System Architecture



4.1 Overview

The system follows a service-oriented architecture. Data flows through the pipeline as follows: market feed → ingestion → TimescaleDB → feature service → model inference → API → frontend.

FastAPI (Ramírez, 2018) [8] handles REST and real-time communication via Socket.IO (Rauch, 2010) [9], while Nginx (Sysoev, 2004) [10] manages routing and static assets. The machine learning subsystem performs sequence forecasting, and the AI layer integrates Gemini API (Google, 2023) [11] and PandasAI (Gabriele, 2023) [12] for query handling and explanation generation.

4.2 Data Pipeline

The data engineering layer is built on PostgreSQL (PostgreSQL Global Development Group, 1996) [13], utilizing TimescaleDB for advanced time-series optimization. The database schema intentionally separates immutable raw market observations, derived feature snapshots, and forecast artifacts into separate logical tables to prevent accidental contamination and enable targeted retention. TimescaleDB hypertables are used to partition data by time while preserving standard SQL semantics. To optimize repetitive queries for dashboard paths, continuous aggregates are implemented to precompute common rollups, such as hourly and daily bars.

4.3 Component Integration

Microservices are justified in this architecture due to heterogeneity in runtime behavior, decoupling network-bound web tasks from GPU-bound training workloads. FastAPI handles I/O-heavy paths asynchronously to reduce thread contention , delegating CPU-bound operations to Celery workers. These Celery workers (Solem, 2009) [14] process asynchronous retraining, historical backfills, and feature recomputation , utilizing Redis as both a cache and the broker backend.

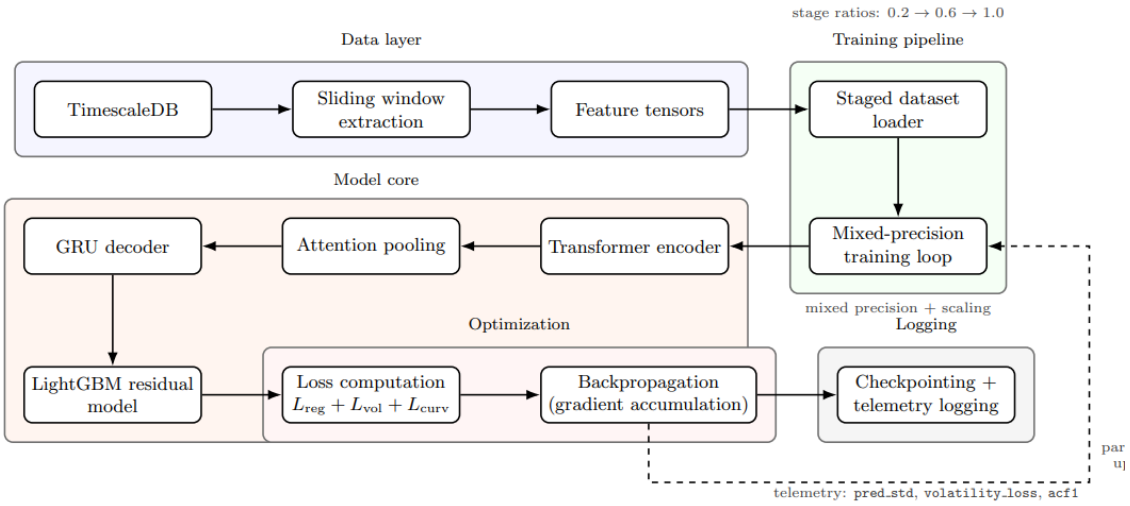
The predictive model and the AI chatbot communicate via a centralized routing logic within the AI layer. The router operates as a deterministic policy stack: schema-dominant questions are routed to structured PandasAI paths, forecast-centric questions are directed to a model interpretation module, and general prompts fall back to the main Gemini path. Finally, this entire service topology is deterministically deployed and managed across environments using Docker Compose (Docker Inc., 2014) [15].

5. Methodology: Predictive Modeling

5.1 Model Architecture

The core forecasting engine utilizes a custom PyTorch (Paszke et al., 2019) [16] sequence model structured around four primary computational stages:

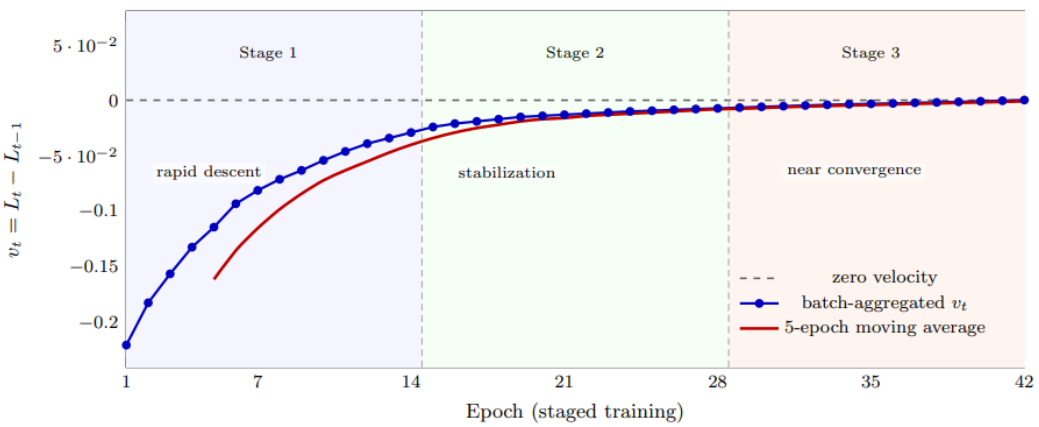
1. A linear embedding layer maps the engineered feature space into the model dimension.
2. Transformer encoder blocks (Vaswani et al., 2017) [17] process the sequence for context-aware interactions, providing global reasoning.
3. Learned attention pooling adaptively aggregates the temporal data, allowing the model to emphasize distinct historical intervals under different market regimes.
4. Finally, GRU temporal refinement smooths the transitions and preserves compact recurrence over the encoded features before a fully connected projection head outputs the final 7-day target.



5.2 Training Process

The model's training loop is highly hardware-aware and tightly controlled to ensure stable convergence:

- **Optimizers and Hyperparameters:** Training employs the AdamW optimizer (Loshchilov & Hutter, 2017) [18] with gradient clipping and staged learning-rate control across three progressive stages. Mixed precision is enabled in CUDA mode to improve tensor-core utilization and reduce memory pressure.
- **Loss Functions:** The primary objective relies on a regression loss optimized for the deployed target. A regime scaling term is used in the weighted training losses, which specifically increases the penalty in high-volatility conditions where forecast errors carry higher operational risk.
- **Overfitting Mitigation:** Overfitting is controlled through a combination of dropout in the encoder blocks, early stopping based on validation deterioration, weight decay, target clipping, and symbol-balanced mini-batches. To further boost robustness, a LightGBM (Ke et al., 2017) [19] ensemble model is trained on tabular features to provide residual corrections, stabilizing outputs during regime shifts where sequence-only models might temporarily miscalibrate.



6. Methodology: AI Chatbot Agent

6.1 Agent Logic

The AI layer bridges the gap between complex stock data and conversational interfaces by seamlessly combining deterministic analytics with generative reasoning. The core agent logic relies on a deterministic policy stack for query routing. Rather than processing every request through a standard LLM, the router dynamically assesses incoming queries to determine the optimal execution path: schema-dominant questions are directed to structured data operations, forecast-centric inquiries are sent to specialized model interpretation modules, and only the remaining general prompts are passed to the Gemini LLM path.

6.2 Knowledge Retrieval

To ensure the chatbot accurately "sees" and interprets the stock data, knowledge retrieval is strictly grounded in deterministic data retrieval. The system utilizes PandasAI to perform schema-grounded analytics. By routing queries that require factual market data directly to structured analytical paths, the system guarantees that the AI agent retrieves real-time, mathematically accurate metrics directly from the underlying database schemas rather than attempting to guess or generate numerical values from its pre-trained weights.

6.3 Prompt Engineering

Prompt engineering within the system is highly systematic and heavily constrained. Prompts are template-driven and explicitly injected with bounded context windows, which include the latest validated market features, underlying model metadata, and strict instructional constraints. The underlying model is explicitly instructed to separate factual data outputs from hypotheses, and it is required to cite the specific source tables or computed fields it utilized during its reasoning process.

6.4 Safety & Guardrails

Preventing hallucinations and unsafe, speculative financial narratives is a critical focus of the agent's design. Hallucination mitigation relies on a combination of retrieval grounding, schema-constrained query generation, and rigorous post-response verification. System-level prompt policies enforce strictly non-speculative behavior for any quantitative outputs; if data is missing or confidence is low, the agent is forced to explicitly return uncertainty or request user clarification. Furthermore, all numerical claims generated by the AI are cross-checked against deterministic query results prior to being released to the user. Any responses failing these verification thresholds are immediately corrected or replaced with deterministic summaries, significantly reducing the risk of the system fabricating financial assertions.

7. Implementation & Tech Stack

The platform relies on a service-oriented architecture designed to separate latency-sensitive network workloads from intensive, GPU-bound machine learning tasks. To ensure deterministic service topology across both development and production environments, the entire deployment is containerized and managed using Docker Compose.

The following table outlines the specific technologies implemented across each layer of the system:

Layer	Technologies Used
Frontend	Static frontend assets served via Nginx ; real-time WebSocket integration via Socket.IO.
Backend	Python, FastAPI (for REST and asynchronous routing); Celery (for asynchronous task queues and backfills).

Database	PostgreSQL; TimescaleDB (for SQL-compliant time-series optimization); Redis (for short-lived caching and Celery brokering).
AI/ML	PyTorch (for the primary sequence model); LightGBM (for tabular ensemble residual corrections); Gemini API (for cloud LLM reasoning); PandasAI (for schema-grounded analytics).
Data Source	vnstock (for fetching market data and historical observations).

8. Evaluation & Results

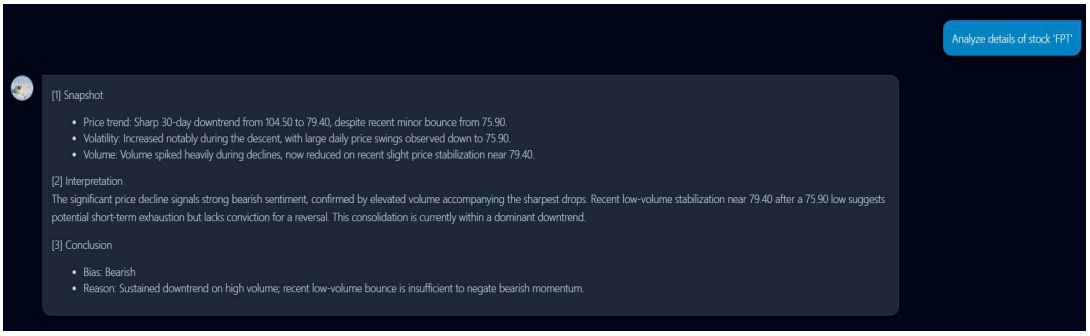
8.1 Predictive Accuracy

The forecasting subsystem's promotion to production requires stable performance across multiple folds rather than just an aggregate average. To emulate deployment chronology, the validation strategy strictly utilizes walk-forward temporal splits, where each fold trains on past data and validates on the next contiguous interval without random time shuffling.

- **Metric 1:** Mean Absolute Error (MAE) is utilized to track the baseline absolute deviation between predicted and actual targets.
- **Metric 2:** Root Mean Squared Error (RMSE) is tracked to measure squared error sensitivity.
- **Additional Metrics:** The system also monitors directional correctness to measure if the predicted return sign aligns with the actual 7-day target.
- **Visuals:** Performance tracking involves comparing the predicted close against actual observations , with specific architectural constraints like a prediction variance floor to ensure the visual and statistical outputs do not collapse into flat lines during low-volatility regimes.

8.2 Agent Performance

- **Response Latency:** End-to-end response time (T_{e2e}) is explicitly decomposed to measure and minimize variance across ingestion, feature materialization, prediction, and transport stages. Predictability of response time is prioritized , and for interactive user paths, bounded micro-batching windows are specifically utilized to protect p95 latency.
- **Contextual Accuracy:** AI responses are strictly evaluated for factual correctness. All numerical claims generated by the agent are cross-checked against deterministic query results before release. Responses that fail these verification thresholds are corrected or replaced with deterministic summaries to ensure the narrative remains aligned with measurable platform state.



8.3 System Usability

While traditional UI testing provides qualitative feedback, the platform's operational usability is heavily quantified through explicit reliability metrics.

- System availability is calculated using Mean Time Between Failures and Mean Time To Recovery: A = \frac{MTBF}{MTBF + MTTR}.
- Service-level objectives enforce strict constraints on latency, error rates, and the freshness of prediction artifacts.
- Real-time usability is further enhanced by Socket.IO channels that deliver task progress and signal updates, utilizing at-least-once delivery semantics and idempotency keys to prevent duplicate UI state transitions.

9. Discussion

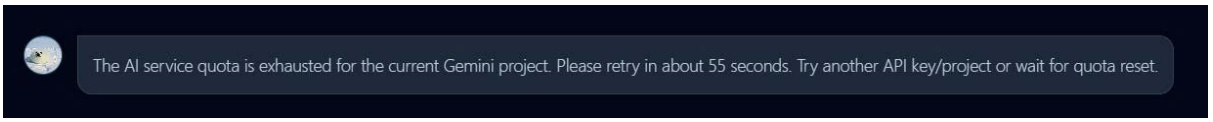
9.1 Analysis of Findings

The platform's performance highlights the advantages of hybrid machine learning architectures in financial forecasting. Financial markets are highly volatile and often exhibit sudden regime shifts, making prediction inherently challenging. The results show that deep learning sequence models, such as Transformers and GRUs, are effective at capturing complex temporal patterns; however, they may become temporarily miscalibrated during periods of high volatility.

To address this limitation, the integration of a LightGBM residual model plays a critical role by acting as a corrective layer over the primary predictions, improving overall robustness. In addition, the inclusion of a regime-aware scaling term in the loss function ensures that errors during high-volatility periods are penalized more heavily. This design aligns the model's optimization process with real-world financial risk considerations, enabling the system to produce more stable and actionable insights from noisy market data.

9.2 Challenges

Developing a responsive AI-powered financial platform introduced several engineering challenges, particularly in managing system performance and operational costs. One major issue involves API rate limits and the cost of cloud-based large language models, as continuous interaction with external services such as the Gemini API can increase both latency and expenses. To mitigate this, the system applies strict retry limits and controlled API usage to maintain efficiency.



When LLM services become unavailable or rate-limited, fallback mechanisms are activated to replace AI-generated responses with deterministic statistical summaries, ensuring continued system functionality. Another challenge lies in handling data noise while maintaining low-latency responses. Redis caching improves performance but introduces a trade-off: longer cache durations increase efficiency but risk serving outdated data, whereas shorter durations ensure freshness at the cost of higher system load. Balancing this trade-off is essential for both reliability and user safety.

9.3 Limitations

Despite its robust architecture, the current system has clear operational and analytical boundaries. The most prominent limitation is its specialization to a single 7-day forecast horizon. While the underlying data schema

and PyTorch pipeline were explicitly designed to support multi-horizon outputs , generalizing the predictions in production introduced conflicting gradients and destabilized model convergence. Collapsing the output to a 7-day scalar target was necessary for mathematical stability and user interpretability, but this restricts the platform's utility for high-frequency day traders or long-term macroeconomic investors. Furthermore, the system is currently constrained to deterministic numerical structures and historical market microstructures.

10. Conclusion & Future Work

Summary

This project successfully met its core objectives by delivering a synchronized platform that effectively bridges the gap between raw data availability and actionable user insights. By unifying robust backend engineering, time-series optimized storage, and mathematically grounded sequence forecasting, the system operates as a production-grade, AI-powered stock analytics platform. It directly addresses the challenge average users face when interpreting complex market data by pairing deterministic forecasting with an intelligent, conversational AI agent. Ultimately, this architecture establishes an operationally credible foundation for algorithmic trading analysis , successfully balancing predictive sophistication with engineering reliability and user trust in the evolving landscape of retail finance.

Future Work

While the current deployment provides a stable and highly capable foundation, future iterations of the platform will focus on expanding its analytical and interactive capabilities. Planned improvements include:

- **Integrating Sentiment Analysis:** Incorporating unstructured data streams from financial news and Twitter to complement the deterministic historical pricing models.
- **Adding "Paper Trading" features:** Allowing users to simulate trades and test the predictive model's 7-day signals in a risk-free environment.
- **Expanding the chatbot to support multiple languages:** Broadening accessibility to make the AI agent's grounded financial reasoning available to a diverse, global retail finance audience.
- **Multi-Horizon Capability:** Expanding the model to provide secondary advisory outputs for 14-day and 30-day horizons once stable calibration can be maintained alongside the primary 7-day operational signal.

References

1. Sanfilippo, S. (2009). *Redis*.
2. Timescale. (2017). *TimescaleDB*.
3. Thinh, V. (2022). *vnstock: Python package for Vietnam stock market*.
4. Box, G. E. P., & Jenkins, G. M. (1970). *Time Series Analysis: Forecasting and Control*.
5. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory.
6. Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation.
7. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.
8. Ramírez, S. (2018). *FastAPI*.
9. Rauch, G. (2010). *Socket.IO*.
10. Sysoev, I. (2004). *Nginx*.
11. Google. (2023). *Gemini: A Family of Highly Capable Multimodal Models*.
12. Gabriele, G. (2023). *PandasAI: Generative AI Python Library*.
13. PostgreSQL Global Development Group. (1996). *PostgreSQL*.
14. Solem, A. (2009). *Celery: Distributed Task Queue*.

15. Docker Inc. (2014). *Docker Compose*.
16. Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library.
17. Vaswani, A., et al. (2017). Attention Is All You Need.
18. Loshchilov, I., & Hutter, F. (2017). Decoupled Weight Decay Regularization (AdamW).
19. Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree.